CHAPTER 2

Running a Rails App in a Container

By now you should be starting to become familiar with the concepts of Docker, like *images*, and running *containers* based on those images. Don't worry if you don't remember all the commands and various options. The most important thing is that you're beginning to understand the high-level concepts—all the rest will follow as you start to use Docker more and more.

In the last chapter, we created our shiny new Rails app. After the amazement began to wear off at how cool it was to generate an app with Ruby supplied by Docker, you were probably left pondering an important question: how the heck do I actually run it?

In this chapter, we're going to pick up where we left off to get our new app up and running. So, have you roasted, ground, brewed, and otherwise prepared the artisanal beverage of your choice? (Just water for me, thanks.) Right, let's crack on…

How Do We Run Our Rails App?

Unfortunately, we can't start a Rails server with just the `ruby:2.6` image—Rails has a few more requirements. For example, we're going to need a JavaScript interpreter (like Node.js) to help with the asset pipeline, plus we'll need to install our gem dependencies. How do we run a Rails server in a container while making sure that these requirements are satisfied?

There are a few approaches we could take. We *could* do what we did in the previous chapter: run `bash` inside a container based on the `ruby:2.6` image, and install what we need from there. However, running the commands *manually* is not easily repeatable. We want a reliable, repeatable way of spinning up Rails servers left, right, and center. Having to run various manual commands every time we want to start a Rails server just isn't going to cut it.